Java MVC Servlet 員工管理系統 教學文件

課程目標

學完本課程後，學生將能夠：

- 理解 MVC（Model-View-Controller）三層架構設計模式
- 使用 JDBC 連接 MySQL 資料庫並執行 CRUD 操作
- 撰寫 Java Servlet 處理 HTTP 請求
- 使用 JSP + JSTL 呈現動態網頁
- 整合以上技術完成完整的員工管理 Web 應用程式

一、MVC 架構概念

```
瀏覽器 (Browser)
    │
    ▼  HTTP Request
┌──────────────────────────────────────────┐
│              Controller (Servlet)        │  ← 接收請求、協調 Model 與 View
└──────────────────────────────────────────┘
    │                        │
    ▼ 呼叫                   ▼ 轉發至
┌──────────────┐        ┌──────────────┐
│  Model       │        │  View (JSP)  │  ← 負責畫面呈現
│  Employee    │        │              │
│  EmployeeDAO │        └──────────────┘
└──────────────┘
    │
    ▼ SQL
┌──────────────┐
│   MySQL DB   │
└──────────────┘
```

層次	類別 / 檔案	職責
Model	`Employee.java`	資料結構（員工物件）
Model	`EmployeeDAO.java`	資料存取邏輯（JDBC）
Controller	`*Servlet.java`	處理請求、呼叫 DAO、轉發 JSP
View	`*.jsp`	HTML 呈現、JSTL 顯示資料

二、Model 層

2.1 Employee.java — 資料模型（JavaBean）

JavaBean 規則：

1. 屬性宣告為 `private`（本範例為 package-level，建議改 `private`）
2. 提供 `getter` / `setter` 方法
3. 有無參數建構子
4. 實作 `toString()`、`equals()`、`hashCode()`

```
public class Employee {
    int employeeNumber;       // 員工編號 (PK)
    String firstName, lastName; // 姓名
    String extension;          // 分機
    String email;              // 電子郵件
    String officeCode;         // 辦公室代號
    int reportsTo;             // 主管編號 (FK)
    String jobTitle;           // 職稱

    // 無參數建構子 (必要)
    public Employee() {}

    // 全參數建構子 (方便快速建立物件)
    public Employee(int employeeNumber, String firstName, String lastName,
                    String extension, String email, String officeCode,
                    int reportsTo, String jobTitle) {
        this.employeeNumber = employeeNumber;
        this.firstName = firstName;
        // ... 其餘省略
    }

    // Getter / Setter (Eclipse 可自動產生: 右鍵 → Source → Generate Getters and
Setters)
    public int getEmployeeNumber() { return employeeNumber; }
    public void setEmployeeNumber(int employeeNumber) { this.employeeNumber =
employeeNumber; }
    // ...
}
```

重要方法說明：

方法	說明
`toString()`	物件轉字串，方便 debug 列印
`hashCode()`	以 email、employeeNumber、firstName、lastName 計算雜湊值
`equals()`	比較兩個 Employee 是否相同（非比較記憶體位址）

```
// equals() 邏輯
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;          // 同一個物件參考
```

```
    if (obj == null) return false;          // null 比較
    if (getClass() != obj.getClass()) return false; // 型別檢查
    Employee other = (Employee) obj;
    return Objects.equals(email, other.email)
        && employeeNumber == other.employeeNumber
        && Objects.equals(firstName, other.firstName)
        && Objects.equals(lastName, other.lastName);
}
```

2.2 EmployeeDAO.java — 資料存取物件（DAO Pattern）

DAO（Data Access Object）將資料庫操作集中管理，讓 Servlet 不需要直接寫 SQL。

2.2.1 資料庫連線

```
public Connection connect() {
    Connection cn = null;
    try {
        // Step 1: 載入 JDBC Driver (MySQL Connector/J)
        Class.forName("com.mysql.cj.jdbc.Driver");

        // Step 2: 建立連線 (URL 格式: jdbc:mysql://主機:埠/資料庫名稱?參數)
        cn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/classicmodels?
useUnicode=true&characterEncoding=utf8",
            "root",   // 帳號
            "1234"    // 密碼
        );
    } catch (SQLException | ClassNotFoundException ex) {
        System.out.println("connect() error:" + ex.getMessage());
    }
    return cn;
}
```

> **注意**：實際開發應將帳號/密碼放入設定檔（如 `web.xml` DataSource 或 properties 檔），不應寫死於程式碼。

2.2.2 查詢所有員工（Read）

```
public List<Employee> getAll() {
    List<Employee> data = new ArrayList<>();
    Connection cn = connect();
    String sql = "select * from classicmodels.employees";
    try {
        Statement st = cn.createStatement();
        ResultSet rs = st.executeQuery(sql);      // 執行查詢，取得結果集
        while (rs.next()) {                        // 逐列讀取
```

```
            int no    = rs.getInt("employeeNumber");
            String fn = rs.getString("firstName");
            // ... 其餘欄位
            Employee e1 = new Employee(no, fn, ln, ex, em, cd, rp, jb);
            data.add(e1);                          // 加入 List
        }
        cn.close();                                // 關閉連線（重要！）
    } catch (SQLException ex) {
        System.out.println("getAll() error:" + ex.getMessage());
    }
    return data;
}
```

流程圖：

```
connect() → createStatement() → executeQuery(sql)
    → while(rs.next()) → 建立 Employee → 加入 List → 回傳 List
```

2.2.3 依編號查詢（使用 Stream API）

```
public Employee findByNumber(int n) {
    List<Employee> data = getAll();
    Employee e1 = data.stream()
                      .filter(e -> e.getEmployeeNumber() == n)  // 過濾條件
                      .findFirst()                               // 取第一筆
                      .get();                                    // 取出值
    return e1;
}
```

> **Lambda 說明**：`e -> e.getEmployeeNumber() == n` 是匿名函式，等同於：
>
> ```
> // 傳統寫法
> for (Employee e : data) {
>  if (e.getEmployeeNumber() == n) return e;
> }
> ```

2.2.4 新增員工（Create — 使用 PreparedStatement）

```
public void insert(Employee p) {
    Connection cn = connect();
    // ? 為參數佔位符，可防止 SQL Injection（資安重要！）
    String sql = "insert into classicmodels.employees(" +
                 "employeeNumber,firstName,lastName,extension," +
                 "email,officeCode,reportsTo,jobTitle) values(?,?,?,?,?,?,?,?)";
```

```
    try {
        cn.setAutoCommit(false);                    // 開啟手動交易
        PreparedStatement st = cn.prepareStatement(sql);
        st.setInt(1, p.getEmployeeNumber());        // 設定第 1 個 ?
        st.setString(2, p.getFirstName());          // 設定第 2 個 ?
        st.setString(3, p.getLastName());
        st.setString(4, p.getExtension());
        st.setString(5, p.getEmail());
        st.setString(6, p.getOfficeCode());
        st.setInt(7, p.getReportsTo());
        st.setString(8, p.getJobTitle());
        st.executeUpdate();                         // 執行 INSERT
        cn.commit();                                // 提交交易
        cn.setAutoCommit(true);
        cn.close();
    } catch (SQLException ex) {
        System.out.println("insert() error:" + ex.getMessage());
    }
}
```

為什麼使用 `PreparedStatement` 而不是 `Statement`?

比較	`Statement`	`PreparedStatement`
SQL Injection	**有風險**	**安全防護**
效能（重複執行）	較差	較佳（預編譯）
程式可讀性	字串拼接，複雜	使用 `?` 佔位符，清晰

三、Controller 層（Servlet）

3.1 Servlet 基本結構

```
@WebServlet("/employees")          // URL 映射（對應網址路徑）
public class EmployeeMVCServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
        // 處理 GET 請求
    }

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {
        // 處理 POST 請求
    }
}
```

3.2 EmployeeMVCServlet — 顯示員工列表

```
@WebServlet("/employees")
public class EmployeeMVCServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        EmployeeDAO dao = new EmployeeDAO();
        List<Employee> emps = dao.getAll();           // 1. 從 DB 取得資料

        request.setAttribute("employees", emps);      // 2. 將資料放入 request
scope

        request.getRequestDispatcher("viewemployee.jsp")
               .forward(request, response);           // 3. 轉發至 JSP
    }
}
```

Request Scope 說明：

- `request.setAttribute("key", value)` → 將資料存入 request
- JSP 中可用 `${key}` 或 `request.getAttribute("key")` 取出
- 生命週期：僅在同一個 request 中有效（forward 可傳遞，redirect 無法）

3.3 AddEmployeeServlet — 新增員工

```
@WebServlet("/AddEmployeeServlet")
public class AddEmployeeServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        // 1. 接收表單參數 (對應 HTML input 的 name 屬性)
        String no = request.getParameter("no");
        String fn = request.getParameter("fn");
        String ln = request.getParameter("ln");
        String ex = request.getParameter("ex");
        String em = request.getParameter("em");
        String cd = request.getParameter("cd");
        String rp = request.getParameter("rp");
        String jb = request.getParameter("jb");

        // 2. 建立 Employee 物件 (String 轉 int 用 Integer.parseInt())
        Employee e1 = new Employee(
            Integer.parseInt(no), fn, ln, ex, em, cd,
            Integer.parseInt(rp), jb
        );
```

```
        // 3. 呼叫 DAO 新增
        EmployeeDAO dao = new EmployeeDAO();
        dao.insert(e1);

        // 4. 重新查詢並顯示最新列表
        List<Employee> emps = dao.getAll();
        request.setAttribute("employees", emps);
        request.getRequestDispatcher("viewemployee.jsp").forward(request,
response);
    }
}
```

3.4 UpdateFormServlet — 顯示修改表單

```
@WebServlet("/updateform")
public class UpdateFormServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        String no = request.getParameter("no");         // 從 URL 取得員工編號
        int n = Integer.parseInt(no);
        EmployeeDAO dao = new EmployeeDAO();
        Employee emp = dao.findByNumber(n);             // 查詢該員工資料
        request.setAttribute("emp", emp);               // 放入 request scope
        request.getRequestDispatcher("updateform.jsp")
               .forward(request, response);             // 轉發至修改表單
    }
}
```

四、View 層（JSP + JSTL）

4.1 viewemployee.jsp — 員工列表頁面

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-
8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<body>
<a href="addemployee.html">新增員工資料</a>
<table border="1" width="70%">
  <thead>
    <td>編號</td><td>姓氏</td><td>名字</td>...
  </thead>
```

```
    <%-- JSTL c:forEach 迴圈輸出每一位員工 --%>
    <c:forEach var="emp" items="${employees}">
      <tr>
        <td>${emp.employeeNumber}</td>
        <td>${emp.firstName}</td>
        <td>${emp.lastName}</td>
        ...
        <td>
          <a href="updateform?no=${emp.employeeNumber}">修改</a>
          <a href="deleteform?no=${emp.employeeNumber}">刪除</a>
        </td>
      </tr>
    </c:forEach>
  </table>
  </body>
  </html>
```

JSTL 核心標籤說明：

標籤	說明	範例
`<c:forEach>`	迴圈	`<c:forEach var="e" items="${list}">`
`<c:if>`	條件判斷	`<c:if test="${x > 0}">`
`<c:choose>`	多條件（switch）	`<c:when test="...">`
`<c:out>`	安全輸出（防 XSS）	`<c:out value="${user.name}"/>`
`${expression}`	EL 表達式	`${emp.firstName}`

4.2 updateform.jsp — 修改員工表單

```
<form action="UpdateEmployeeServlet" method="post">
   員工編號:<input type="text" name="no" value="${emp.employeeNumber}"/><br/>
   姓氏:<input type="text" name="ln" value="${emp.lastName}"/><br/>
   名字:<input type="text" name="fn" value="${emp.firstName}"/><br/>
   分機號碼:<input type="text" name="ex" value="${emp.extension}"/><br/>
   電子郵件:<input type="text" name="em" value="${emp.email}"/><br/>
   辦公室代號:<input type="text" name="cd" value="${emp.officeCode}"/><br/>
   主管編號:<input type="text" name="rp" value="${emp.reportsTo}"/><br/>
   職位:<input type="text" name="jb" value="${emp.jobTitle}"/><br/>
   <input type="submit"/>
</form>
```

EL（Expression Language）取值規則：

- `${emp.employeeNumber}` → 等同呼叫 `emp.getEmployeeNumber()`
- JSP EL 會自動呼叫對應的 getter 方法
- 屬性名稱首字母小寫（`emp.firstName` → `getFirstName()`）

五、完整操作流程

5.1 查詢員工列表流程

```
1. 使用者在瀏覽器輸入 http://localhost:8080/專案名/employees
2. Tomcat 找到 @WebServlet("/employees") → EmployeeMVCServlet
3. doGet() 執行:
   a. EmployeeDAO.getAll() → 連接 MySQL → 執行 SELECT → 回傳 List<Employee>
   b. request.setAttribute("employees", emps)
   c. forward 到 viewemployee.jsp
4. JSP 以 c:forEach 迴圈輸出 HTML 表格
5. 瀏覽器顯示員工列表
```

5.2 新增員工流程

```
1. 點選「新增員工資料」→ addemployee.html (靜態表單)
2. 填寫資料後按送出 (POST /AddEmployeeServlet)
3. AddEmployeeServlet.doPost() 執行:
   a. request.getParameter() 取得表單資料
   b. 建立 Employee 物件
   c. EmployeeDAO.insert() → 執行 INSERT SQL
   d. 重新查詢並 forward 到 viewemployee.jsp
4. 顯示更新後的員工列表
```

5.3 修改員工流程

```
1. 點選列表中的「修改」連結
   → GET /updateform?no=1002
2. UpdateFormServlet.doGet() 執行:
   a. 取得 no 參數
   b. EmployeeDAO.findByNumber(n) 查詢員工
   c. request.setAttribute("emp", emp)
   d. forward 到 updateform.jsp
3. updateform.jsp 預填表單資料
4. 使用者修改後送出 → POST /UpdateEmployeeServlet
5. UpdateEmployeeServlet 執行更新並回到列表
```

六、關鍵技術重點整理

6.1 JDBC 操作步驟

```
1. Class.forName()      → 載入 Driver
2. DriverManager.getConnection() → 建立 Connection
3. cn.createStatement() 或 cn.prepareStatement() → 建立 Statement
```

```
4. st.executeQuery()    → 查詢（回傳 ResultSet）
   st.executeUpdate()   → 新增/修改/刪除（回傳影響列數）
5. rs.next() / rs.getString() → 讀取結果
6. cn.close()           → 關閉連線（必要！）
```

6.2 Servlet 生命週期

```
載入 → init() → service() → doGet() / doPost() → destroy()
                  ↑
          每次 HTTP 請求都會呼叫
```

6.3 forward vs redirect

比較	forward	redirect
URL 變化	不變	改變
request 資料	可傳遞	**無法傳遞**
用法	`RequestDispatcher.forward()`	`response.sendRedirect()`
適用場景	顯示查詢結果	新增/刪除後防止重複提交

七、練習題

基礎題

1. 在 `EmployeeDAO` 中新增 `update(Employee p)` 方法，執行 UPDATE SQL 更新員工資料。
2. 完成 `UpdateEmployeeServlet.doPost()` 方法，取得表單資料後呼叫 `dao.update()` 更新資料。

進階題

3. 新增 `DeleteEmployeeServlet`，對應 `/deleteform` URL，接收員工編號後執行刪除，再轉回列表頁面。
4. 在 `EmployeeDAO` 中新增 `findByName(String name)` 方法，以姓名搜尋員工（使用 `LIKE` SQL）。
5. 在 `viewemployee.jsp` 中加入搜尋表單，讓使用者可依姓名搜尋員工。

挑戰題

6. 改良 `connect()` 方法，將資料庫 URL、帳號、密碼移至設定（避免硬編碼安全風險）。
7. 在 `AddEmployeeServlet` 中加入輸入驗證：員工編號不得為空且需為正整數，電子郵件需符合格式。

八、參考：update() 方法實作範例

```java
public void update(Employee p) {
    Connection cn = connect();
    String sql = "UPDATE classicmodels.employees " +
```

```
                    "SET firstName=?, lastName=?, extension=?, " +
                    "email=?, officeCode=?, reportsTo=?, jobTitle=? " +
                    "WHERE employeeNumber=?";
    try {
        cn.setAutoCommit(false);
        PreparedStatement st = cn.prepareStatement(sql);
        st.setString(1, p.getFirstName());
        st.setString(2, p.getLastName());
        st.setString(3, p.getExtension());
        st.setString(4, p.getEmail());
        st.setString(5, p.getOfficeCode());
        st.setInt(6, p.getReportsTo());
        st.setString(7, p.getJobTitle());
        st.setInt(8, p.getEmployeeNumber());    // WHERE 條件
        st.executeUpdate();
        cn.commit();
        cn.setAutoCommit(true);
        cn.close();
    } catch (SQLException ex) {
        System.out.println("update() error:" + ex.getMessage());
    }
}
```